Lab 06 — Hands-on lab: making data outlive its container

Goal: lose data once to understand why, then make it survive with a named volume, a bind mount and a tmpfs; see with your own eyes who owns the files under rootless; and back up, then restore, a PostgreSQL database in two ways.

Prerequisites — Labs 01 to 05 completed. The `alpine` and `nginx:alpine` images are present; `postgres:16-alpine` gets pulled in step 2 (297 MB).

Files provided (`files/`) — none: everything is created at the terminal.

The output below comes from an Ubuntu/WSL with the standard `subuid` range (`100000:65536`). Container IDs, dates and hashes will differ on your machine.

Step 1 — Losing a piece of data

```
mkdir -p ~/labo-docker/06 && cd ~/labo-docker/06
podman run --name scribe alpine sh -c 'echo "Monday note" > /data.txt; ls -l /data.txt'
podman diff scribe
```

Observe `-rw-r--r-- 1 root root 12 … /data.txt`, then `A /data.txt` and `C /etc`: the writable layer of the `scribe` container holds one added file and one modified directory.

```
podman run --rm alpine ls /data.txt
podman start -a scribe
podman rm scribe
```

Observe `ls: /data.txt: No such file or directory` for the new container: the image has not changed. Then the restarted `scribe` does show its file: `stop` / `start` keep the layer. Then `rm`: the file no longer exists anywhere.

Explanation. Each container has its own writable layer, invisible to the others and destroyed with it. Data that must survive cannot live there.

Step 2 — A named volume for PostgreSQL

```
podman volume create pgdata
podman volume inspect pgdata
```

Observe `"Mountpoint": "/home/<you>/.local/share/containers/storage/volumes/pgdata/_data"`, `"NeedsCopyUp": true`, `"MountCount": 0`: an empty directory, managed by Podman, mounted nowhere yet.

```
podman run -d --name db -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman logs db 2>&1 | tail -1
podman exec db psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT INTO
clients(nom) VALUES ('Durand'),('Peeters');"
podman exec db psql -U postgres -c "SELECT * FROM clients;"
podman inspect db --format '{{json .Mounts}}'
```

Observe `database system is ready to accept connections`, `CREATE TABLE` / `INSERT 0 2`, the two rows, then the mount: `"Type":"volume","Name":"pgdata",…` `"Destination":"/var/lib/postgresql/data","RW":true`.

```
podman rm -f -t 0 db
podman volume ls
podman run -d --name db2 -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman exec db2 psql -U postgres -c "SELECT * FROM clients;"
podman rm -f -t 0 db2
```

Observe `pgdata` still listed after the container was removed, and the second container finding `Durand` and `Peeters` again.

Explanation. The container is gone; the volume stayed. The second container found an already-initialised `PGDATA` and recreated nothing — `POSTGRES_PASSWORD` only matters on the very first start.

Step 3 — The anonymous volume you never asked for

```
podman image inspect postgres:16-alpine --format '{{.Config.Volumes}}'
podman run -d --name anon -e POSTGRES_PASSWORD=secret postgres:16-alpine
sleep 3 ; podman volume ls
podman inspect anon --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.Destination}}{{end}}'
podman rm -f -t 0 anon
podman volume ls
```

Observe `map[/var/lib/postgresql/data:{}]`: the image declares a `VOLUME`. Without `-v`, Podman created a volume with a 64-hex-character name, mounted at the same place. After `rm`, it is **still there**.

```
podman volume ls --filter dangling=true
```

Observe that the filter lists the anonymous volume… **and** `pgdata`. "Dangling" means "mounted by no existing container" — and since `db2` was removed, that applies to `pgdata`, which nevertheless holds your clients. Feeding this list to `volume rm` (or running `podman volume prune`) would destroy both.

```
podman volume rm $(podman volume ls --filter dangling=true -q | grep -v '^pgdata$')
podman run -d --name anon2 -e POSTGRES_PASSWORD=secret postgres:16-alpine
sleep 3 ; podman rm -f -t 0 -v anon2
podman volume ls
```

Observe that only `pgdata` remains: the first anonymous volume was removed by name, the second together with its container thanks to `rm -v`.

Explanation. `podman rm` does not remove volumes; `podman rm -v` removes the container's **anonymous** volumes (never the named ones). The `dangling=true` filter cannot tell garbage from a precious volume waiting between two containers: read the list, then delete by name.

Step 4 — Who owns the files?

```
ls -ln ~/.local/share/containers/storage/volumes/
ls ~/.local/share/containers/storage/volumes/pgdata/_data
```

Observe `drwx------ 3 1000 1000 … pgdata`: the volume's directory is yours. But its content: `ls: cannot open directory … Permission denied`.

```
podman unshare cat /proc/self/uid_map
podman unshare ls -ln ~/.local/share/containers/storage/volumes/pgdata/_data | head -4
```

Observe the table `0 1000 1` then `1 100000 65536`, and the files `-rw------- 1 70 70 … PG_VERSION`: inside the namespace, they belong to UID 70, `postgres`.

Explanation. PostgreSQL writes as UID 70. Outside the namespace, that UID translates to `100000 + 70 - 1 = 100069`: a user who is not you. `podman unshare` runs a command *inside* the namespace, with the same view as the container. (In the sandbox these labs were written in, the subuid range starts at 1001; the principle is identical.)

Step 5 — A bind mount for nginx

```
mkdir -p html && echo '<h1>Version 1</h1>' > html/index.html
podman run -d --name web -p 18080:80 -v ./html:/usr/share/nginx/html:ro nginx:alpine
sleep 1 ; curl -s localhost:18080
echo '<h1>Version 2</h1>' > html/index.html
curl -s localhost:18080
```

Observe `<h1>Version 1</h1>` then `<h1>Version 2</h1>`: the file modified on the host is served immediately, with no restart and no rebuild.

```
podman exec web sh -c 'echo "<h1>pirate</h1>" > /usr/share/nginx/html/index.html'
podman inspect web --format '{{json .Mounts}}'
```

Observe `sh: can't create …/index.html: Read-only file system`, and in the inspect output `"Type":"bind","Source":"/home/<you>/labo-docker/06/html",… "RW":false`.

```
podman run -d --name web-rw -p 18081:80 -v ./html:/usr/share/nginx/html nginx:alpine
podman exec web-rw sh -c 'echo "<h1>Version 3 (written from the container)</h1>" >
/usr/share/nginx/html/index.html'
cat html/index.html ; ls -ln html/
podman cp web:/etc/nginx/nginx.conf ./nginx.conf && head -3 nginx.conf
podman rm -f -t 0 web web-rw
```

Observe `Version 3` in **your** file, which still belongs to `1000 1000`: written by the container's root, which is you. And `podman cp` pulled the nginx configuration out without any mount.

Explanation. A bind mount is the same directory seen from two places. `:ro` protects your files; without it, the container writes into your folder with the UID matching its user.

Step 6 — The UID trap

```
mkdir -p output
podman run --rm -v ./output:/out alpine sh -c 'id -u; echo hello > /out/root.txt'
ls -ln output
podman run --rm --user 1000 -v ./output:/out alpine sh -c 'id -u; echo hello > /out/user.txt'
```

Observe `0` and a `root.txt` file owned by `1000 1000`. Then `1000` and `sh: can't create /out/user.txt: Permission denied`.

```
podman run --rm --user 1000 -v ./output:/out:U alpine sh -c 'id -u; echo hello > /out/user.txt;
ls -ln /out'
ls -ln output
```

Observe inside the container `-rw-r--r-- 1 1000 0 … user.txt`, and on the host `-rw-r--r-- 1 100999 1000 … root.txt` and `user.txt`: `:U` changed the owner **of the whole folder** to the container's UID, which is no longer you.

```
cat output/user.txt
podman unshare chown -R 0:0 output && ls -ln output
```

Observe that reading works (`r` permission for everyone), then `1000 1000`: the folder is yours again.

Explanation. The container's root is you; the container's UID 1000 is 100999 on the host. `:U` performs a real, persistent `chown`; `podman unshare chown 0:0` undoes it, since root inside the namespace = 1000 outside.

Step 7 — tmpfs and the `--mount` syntax

```
podman run --rm --tmpfs /scratch:size=64m alpine sh -c 'df -h /scratch; echo temporary >
/scratch/f.txt; ls /scratch'
podman run --rm --mount type=tmpfs,destination=/scratch,tmpfs-size=64m alpine df -h /scratch
```

Observe `tmpfs 64.0M 0 64.0M 0% /scratch` and `f.txt`: a 64 MB in-memory disk that is already gone on the next line.

```
podman run --rm --mount type=volume,source=pgdata,destination=/data alpine ls /data | head -3
podman run --rm --mount type=bind,source=./html,destination=/html,ro alpine sh -c 'cat
/html/index.html; touch /html/x'
podman run --rm -v ./html:/html:ro,Z alpine cat /html/index.html
```

Observe `PG_VERSION`, `base`, `global` (the volume as seen by another container), `Read-only file system` on the `touch`, and `:Z` accepted without changing anything.

Explanation. `--mount` states explicitly what you mount; `-v` guesses from the shape of the source. `:Z` does nothing without SELinux, but gets your commands ready for RHEL servers.

Step 8 — Back up, break, restore

```
podman run -d --name db -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman exec db pg_dump -U postgres > dump.sql
ls -lh dump.sql ; grep -A3 "COPY public.clients" dump.sql
podman volume rm pgdata
```

Observe a `dump.sql` of about 1.8 kB containing `COPY public.clients (id, nom) FROM stdin;` followed by the two rows: a logical backup, taken while the database runs. Then `Error: volume pgdata is being used by the following container(s): … volume is being used`.

```
podman stop -t 5 db
podman volume export pgdata -o pgdata.tar
ls -lh pgdata.tar ; tar tf pgdata.tar | head -3
podman volume create pgdata-copy
podman volume import pgdata-copy pgdata.tar
podman run -d --name db-copy -e POSTGRES_PASSWORD=secret -v pgdata-
copy:/var/lib/postgresql/data postgres:16-alpine
sleep 5 ; podman exec db-copy psql -U postgres -c "SELECT * FROM clients;"
```

Observe a 39 MB archive (`PG_VERSION`, `base/`, …) and, in the imported volume, the two clients: a physical backup, container stopped.

```
podman volume create pgdata-blank
podman run -d --name db-blank -e POSTGRES_PASSWORD=secret -v pgdata-
blank:/var/lib/postgresql/data postgres:16-alpine
sleep 5 ; podman exec db-blank psql -U postgres -c "SELECT * FROM clients;"
podman exec -i db-blank psql -U postgres -q < dump.sql
podman exec db-blank psql -U postgres -c "SELECT * FROM clients;"
```

Observe `ERROR: relation "clients" does not exist` on the blank database; replaying the dump prints only two small results (`set_config`, `setval`); then the two clients are back.

```
podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .
ls -lh pgdata.tgz
podman system df -v | sed -n '/Local Volumes/,$p'
```

Observe `pgdata.tgz` (4.5 MB compressed) written into your folder by a throwaway container, and the three volumes of about 40 MB each.

Explanation. `pg_dump` reads a consistent snapshot while the database runs; copying files requires a stop. `volume export` / `import` are Podman-specific; the Alpine container with two mounts is the method that works everywhere — and the only one that reads UID 100069's files without `unshare`.

Clean-up

```
podman rm -f -t 0 db db-copy db-blank
podman volume rm pgdata pgdata-copy pgdata-blank
podman volume ls
podman rmi docker.io/library/postgres:16-alpine
rm -rf ~/labo-docker/06/output ~/labo-docker/06/pgdata.tar ~/labo-docker/06/pgdata.tgz
```

Observe an empty `podman volume ls`. Keep `html/` and `dump.sql`: later labs do not need them, but they take up 2 kB. `postgres:16-alpine` comes back in lab 07 — removing it here saves 297 MB until then, and lab 07 removes it again.

What you must be able to state now

- `stop` / `start` keep the writable layer; `rm` destroys it: `No such file or directory`.
- A named volume outlives its container: `db2` finds the clients of `db`.
- `VOLUME` in the image = a 64-character anonymous volume if you forget `-v`; `rm -v` takes it along, and `dangling=true` also lists your named volumes that are idle.
- PostgreSQL's files belong to `70` inside the namespace and `100069` outside: `Permission denied` without `podman unshare`.
- A `:ro` bind mount is served live and refuses writes: `Read-only file system`.
- `--user 1000` cannot write into your folder; `:U` hands it over, `podman unshare chown` takes it back.
- `pg_dump` while running, `volume export` or `tar` with the container stopped; `volume rm` refuses while a container mounts the volume.